

## Practical No 2

Aim:

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI Version:

Code:

```
import random
```

```
import math
```

```
def is_prime(num):
```

```
    if num < 2: return False
```

```
    for i in range(2, int(math.sqrt(num)) + 1):
```

```
        if num % i == 0: return False
```

```
    return True
```

```
def generate_prime(min_val=100, max_val=999):
```

```
    prime = random.randint(min_val, max_val)
```

```
    while not is_prime(prime):
```

```
        prime = random.randint(min_val, max_val)
```

```
    return prime
```

```
def extended_gcd(a, b):
```

```
    if a == 0: return (b, 0, 1)
```

```
    g, y, x = extended_gcd(b % a, a)
```

```
    return (g, x - (b // a) * y, y)
```

```
def mod_inverse(e, phi):
```

```
    g, x, y = extended_gcd(e, phi)
```

```
    if g != 1:
```

```
        raise Exception('Modular inverse does not exist')
```

```
    return x % phi
```

```
def generate_keypair():
```

```
    p = generate_prime()
```

```
    q = generate_prime()
```

```
    while p == q:
```

```
        q = generate_prime()
```

```
    n = p * q
```

```
    phi = (p - 1) * (q - 1)
```

```
    e = random.randrange(2, phi)
```

```
    g = math.gcd(e, phi)
```

```

while g != 1:
    e = random.randrange(2, phi)
    g = math.gcd(e, phi)

d = mod_inverse(e, phi)
return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    e, n = public_key
    return [pow(ord(char), e, n) for char in plaintext]

def decrypt(private_key, ciphertext):
    d, n = private_key
    return "".join([chr(pow(char, d, n)) for char in ciphertext])

if __name__ == '__main__':
    print("--- RSA Encryption/Decryption CLI ---")
    print("Generating RSA keys...")
    public, private = generate_keypair()
    print(f"Public Key: {public}")
    print(f"Private Key: {private}\n")

    while True:
        message = input("Enter a message to encrypt (or type 'exit' to quit): ")

        if message.strip().lower() == 'exit':
            print("Exiting the RSA tool. Goodbye!")
            break

        if not message:
            print("Please enter a valid message.\n")
            continue

        encrypted_msg = encrypt(public, message)
        print(f"\nEncrypted Message (Ciphertext array): {encrypted_msg}")

        decrypted_msg = decrypt(private, encrypted_msg)
        print(f"Decrypted Message: {decrypted_msg}")
        print("-" * 50 + "\n")

```

Output:

```

Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/Omith/OneDrive/Documents/Cyber & Information Security/Practical No 2 Cyber
& Information Security T097.py
--- RSA Encryption/Decryption CLI ---
Generating RSA keys...
Public Key: (497093, 944159)
Private Key: (848357, 944159)

Enter a message to encrypt (or type 'exit' to quit): OMITH T097

Encrypted Message (Ciphertext array): [731179, 783525, 743126, 703372, 580475, 807073, 703372, 777
020, 706082, 192053]
Decrypted Message: OMITH T097
-----

Enter a message to encrypt (or type 'exit' to quit):

```

GUI Version:

Code:

```

import flet as ft
import random
import math

```

# --- RSA Core Logic ---

```

def is_prime(num):
    if num < 2: return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0: return False
    return True

```

```

def generate_prime(min_val=100, max_val=999):
    prime = random.randint(min_val, max_val)
    while not is_prime(prime):
        prime = random.randint(min_val, max_val)
    return prime

```

```

def extended_gcd(a, b):
    if a == 0: return (b, 0, 1)
    g, y, x = extended_gcd(b % a, a)
    return (g, x - (b // a) * y, y)

```

```

def mod_inverse(e, phi):
    g, x, y = extended_gcd(e, phi)
    return x % phi

def generate_keys():
    p, q = generate_prime(), generate_prime()
    while p == q: q = generate_prime()
    n = p * q
    phi = (p - 1) * (q - 1)
    e = random.randrange(2, phi)
    while math.gcd(e, phi) != 1:
        e = random.randrange(2, phi)
    d = mod_inverse(e, phi)
    return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    e, n = public_key
    return [pow(ord(char), e, n) for char in plaintext]

def decrypt(private_key, ciphertext):
    d, n = private_key
    return ''.join([chr(pow(char, d, n)) for char in ciphertext])

# --- Modern Flet GUI Interface ---
def main(page: ft.Page):
    # Modern Window Settings
    page.title = "RSA Cryptography Studio"
    page.window.width = 550
    page.window.height = 750
    page.theme_mode = ft.ThemeMode.DARK
    page.bgcolor = "#0D1117" # Deep sleek dark background
    page.padding = 30
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    # Application State
    public_key, private_key = generate_keys()
    current_ciphertext = []

    # Custom Color Palette
    ACCENT_CYAN = "#00FFA3"
    CARD_BG = "#161B22"
    TEXT_MUTED = "#8B949E"

```

```
# --- UI Components ---
```

```
# 1. Header
```

```
header = ft.Row(
    controls=[
        ft.Icon(ft.Icons.SHIELD_MOON, color=ACCENT_CYAN, size=32),
        ft.Text("RSA Studio", size=28, weight=ft.FontWeight.W_800, color=ft.Colors.WHITE)
    ],
    alignment=ft.MainAxisAlignment.CENTER
)
```

```
# 2. Key Display Card
```

```
key_display = ft.Column(
    controls=[
        ft.Text("CURRENT KEYS", size=12, weight=ft.FontWeight.BOLD, color=TEXT_MUTED),
        ft.Text(f"Public Key: {public_key}", font_family="monospace", color=ACCENT_CYAN,
size=14),
        ft.Text(f"Private Key: {private_key}", font_family="monospace", color="#FF7B72",
size=14),
    ],
    spacing=5
)
```

```
key_card = ft.Container(
    content=key_display,
    bgcolor=CARD_BG,
    padding=20,
    border_radius=15,
    width=float("inf")
)
```

```
# 3. Input/Output Fields
```

```
input_box = ft.TextField(
    label="Enter message to process...",
    multiline=True,
    min_lines=3,
    max_lines=4,
    border_color="#30363D",
    focused_border_color=ACCENT_CYAN,
    cursor_color=ACCENT_CYAN,
    border_radius=12
)
```

```
output_box = ft.TextField(
```

```

label="Terminal Output",
multiline=True,
min_lines=5,
max_lines=8,
read_only=True,
border_color="#30363D",
focused_border_color="#A371F7", # Soft purple for output
text_style=ft.TextStyle(font_family="monospace"),
border_radius=12
)

# --- Event Handlers ---
def show_toast(message, color):
    page.overlay.append(
        ft.SnackBar(
            ft.Text(message, color=ft.Colors.WHITE, weight=ft.FontWeight.BOLD),
            bgcolor=color,
            open=True
        )
    )
    page.update()

def handle_encrypt(e):
    nonlocal current_ciphertext
    message = input_box.value
    if not message:
        show_toast("⚠ Enter a message first!", ft.Colors.RED_700)
        return

    current_ciphertext = encrypt(public_key, message)
    output_box.value = str(current_ciphertext)
    show_toast("🔒 Message Encrypted", "#238636")
    page.update()

def handle_decrypt(e):
    if not current_ciphertext:
        show_toast("⚠ No encrypted data in memory!", ft.Colors.RED_700)
        return

    decrypted_message = decrypt(private_key, current_ciphertext)
    output_box.value = decrypted_message
    show_toast("🔓 Message Decrypted", "#8957E5")
    page.update()

```

```

def handle_new_keys(e):
    nonlocal public_key, private_key, current_ciphertext
    public_key, private_key = generate_keys()

    # Update the text inside the column inside the container
    key_display.controls[1].value = f"Public Key: {public_key}"
    key_display.controls[2].value = f"Private Key: {private_key}"

    input_box.value = ""
    output_box.value = ""
    current_ciphertext = []
    show_toast("🌟 New RSA keys generated!", ACCENT_CYAN)
    page.update()

# --- Action Buttons ---
btn_style = ft.ButtonStyle(
    shape=ft.RoundedRectangleBorder(radius=10),
    padding=15
)

buttons_row = ft.Row(
    controls=[
        ft.ElevatedButton("ENCRYPT", icon=ft.Icons.LOCK, on_click=handle_encrypt,
                           bgcolor="#238636", color=ft.Colors.WHITE, style=btn_style, expand=True),
        ft.ElevatedButton("DECRYPT", icon=ft.Icons.LOCK_OPEN, on_click=handle_decrypt,
                           bgcolor="#8957E5", color=ft.Colors.WHITE, style=btn_style, expand=True),
    ],
    alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
)

reset_button = ft.TextButton(
    "Generate New Keys",
    icon=ft.Icons.REFRESH,
    icon_color=ACCENT_CYAN,
    on_click=handle_new_keys,
    style=ft.ButtonStyle(color=ACCENT_CYAN)
)

# Build the Page Layout
page.add(
    header,
    ft.Divider(height=20, color="transparent"),
    key_card,
    reset_button,

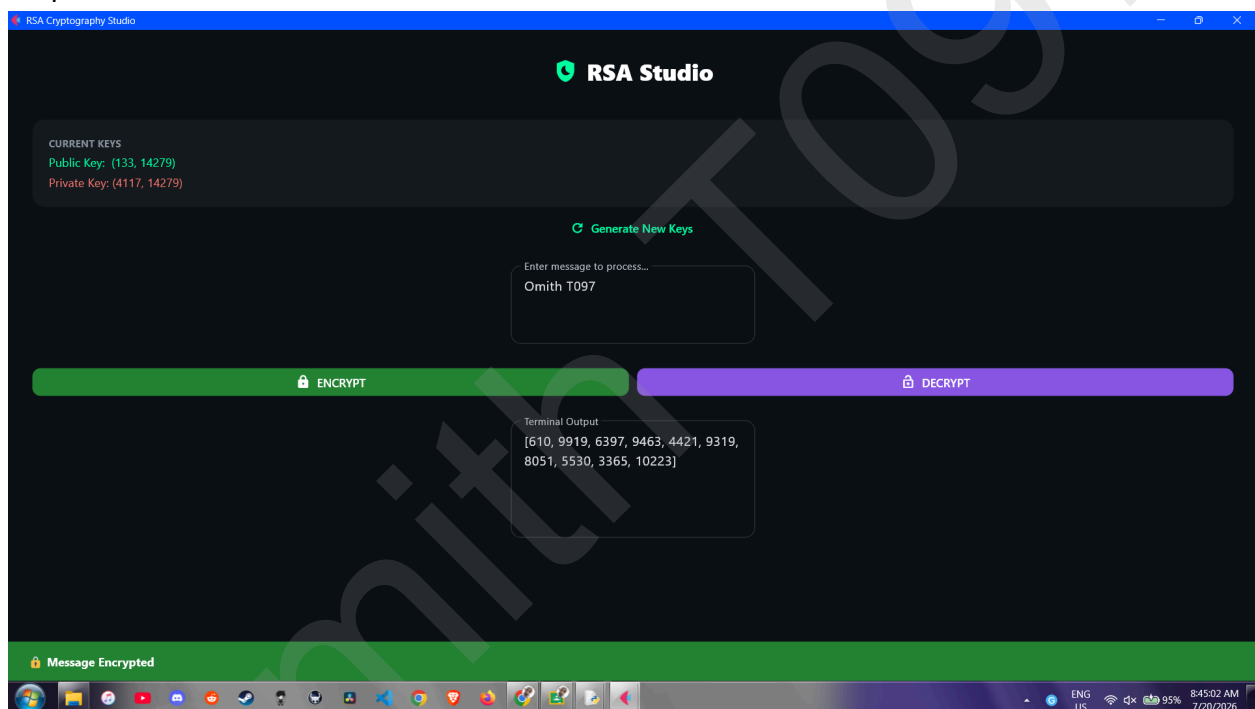
```

## SHETH LUJ AND SIR MV COLLEGE

```
ft.Divider(height=10, color="transparent"),
input_box,
ft.Divider(height=10, color="transparent"),
buttons_row,
ft.Divider(height=10, color="transparent"),
output_box
)

# Run the app
if __name__ == "__main__":
    ft.run(main)
```

Output:



# SHETH LUJ AND SIR MV COLLEGE

